

ROS 2 as a testable robot software system

M12 • 12 Sep 2027 - 14 Oct 2027 • 75 hours

This month's deliverable

You will combine individual models, drivers and controllers into a reproducible multi-node system. The final telemetry-and-control stack v1.0 includes custom interfaces where needed, specified QoS, separated callbacks, coordinate transforms at the correct time, a software simulator of the hardware with lifecycle management, diagnostics and replay of a recorded fault. This is the software foundation for the legs and hand; one leg and one representative finger in separate namespaces are sufficient for now.

Use this architecture: an IMU or joint data source publishes data; an estimator outputs state; a controller produces a bounded request; the hardware layer executes authorised commands; a monitor checks freshness and state; and a recorder saves the experiment. One component owns each physical resource. Activate control after checking readiness; the presence of a topic, a message publication channel, in the graph does not establish sensor or actuator readiness.

Environment and prerequisites

The main programme route uses ROS 2 Jazzy on Ubuntu 24.04. Packages support amd64 and arm64; on an Apple Silicon MacBook, use an ARM64 Linux environment. State the virtualisation method, network settings and device access. Test DDS participant discovery, USB passthrough to the guest, graphics and tracing in a small experiment: their operation depends on the virtual environment. If the graphical interface causes problems, first run the system from the console and record data, then state graphics and physical-hardware limitations separately.

Previous months must provide stable telemetry fields, units, monotonic measurement timing, controller and state-estimation models, and negative tests. Do not rewrite the mathematics when moving to ROS. Extract validated calculations into a library and call it from a small node so the test without ROS continues to work. Use C++ for execution-time measurement and the hardware layer, and Python for scenarios and analysis; a third language is optional.

Completion criteria

The system builds and launches from its documentation; the graph and interface ownership are clear; QoS and execution choices are justified by measurements. The simulator activates and deactivates correctly, and a recorded failure can be replayed. Prepare material for the architectural review and recording replay at checkpoint G2. Its integration hours are separate from these 75 hours. A simulator running in ROS does not close the separate HIL criteria involving real hardware. State every unconfirmed property in the report.

Organising the 75 hours

Four work blocks

Block A, hours 1-20: graph, messages and launch. Block B, 21-40: DDS, QoS and callbacks. Block C, 41-60: TF2, URDF and hardware-component lifecycle. Block D, 61-75: diagnostics, recording and replay. Each of the first three blocks contains 10 hours of theory and analysis and 10 hours of practice; the fourth contains 5 hours of analysis and 10 hours of practice. Total: $35 + 40 = 75$ hours. The work steps on pages 9-10 expand these same hours and require no additional time allocation.

Divide each ten-hour theory block into five two-hour sessions: 20 minutes reviewing the previous model, 45 minutes reading the relevant section, 40 minutes deriving a small example or examining a log, and 15 minutes recording the conclusion. Also divide the ten-hour Sunday practical session into five two-hour periods with breaks between them. Breaks are excluded from net study hours. Allocate the final five hours of analysis as $2 + 2 + 1$.

Start with the question, then choose the tool

The main environment is ROS 2 Jazzy on Ubuntu 24.04, colcon, rclcpp for C++ and rclpy for Python; also use rosbag2, ros2_control, tf2 and one plotting tool. Begin each session with a testable question: how does the input affect the output, which state cannot be observed, and which data reveal its changes? Before running, write down the expected sign, scale or sequence of events. Compare the prediction with the result and examine at least one discrepancy. Accompany every plot with an explanation of the observed relationship.

Maintain one repository with model, config, experiments, tests and reports directories. Every experiment needs a unique identifier, code version, configuration, parameter source, initial state, units and raw data. The analysis script reads a saved experiment and builds the report without manually correcting measurements. The final ROS 2 telemetry-and-control stack v1.0 project combines telemetry and control; individual exercises gradually form its test suite.

Time accounting and reducing scope

Calendar boundaries may coincide with an adjacent module. Keep rest days, holiday intervals and integration checks as specified in the overall calendar; place numbered steps in available time without counting hours twice. Do not add hours to this module. Assess equipment and library readiness at the outset rather than spending the practical session on an unplanned platform change.

If you fall behind, reduce the dashboard to four plots and retain one hardware simulator and one way of composing nodes in a process. Preserve the interface-ownership table, QoS experiment, callback-delay measurements, TF at measurement time, lifecycle check and automated fault replay. A second visualisation tool and porting nodes to Rust are optional. If tracing or a physical driver is unavailable, name the specific untested criterion; architectural validation on a simulator does not replace HIL.

Graph, interfaces and launch

Study how nodes interact

A topic suits an IMU or joint-state stream; a service suits a short configuration or state request; an action suits a long-running motion with feedback and cancellation. A parameter stores a setting but does not replace a reference stream on every cycle. Study nodes, topics, services, actions and parameters, then launch, namespaces and composition of nodes within one process. Composition changes data transfer and failure consequences; the number of logical components need not equal the number of processes.

First use standard `sensor_msgs/Imu` and `JointState` messages wherever they express the intended meaning accurately. A custom message is needed for fields missing from the interface description: command identifier, failure reason, state version or associated quality flag. Study conversion of `.msg`, `.srv` and `.action` to IDL, bounded arrays and strings, and serialisation. State units and coordinate frames for every quantity, and the time source for timestamp. Version semantic changes, as well as package names.

Task 1. Interfaces for one leg and finger

Inputs: definitions from M11, one leg with known joint names and one software finger simulator. 1) Draw a graph containing source, estimator, controller and monitor. 2) Create a field-ownership table. 3) Create an interface package and two minimal publishing and checking nodes. 4) Launch `/leg` and `/hand` in separate namespaces. Deliverable: an interface document, messages and a working example. Check: angles in rad, gyroscope angular velocity in rad/s and acceleration in m/s²; map joints by name rather than arbitrary array position. Error to detect: graph inspection reveals two publishers simultaneously setting the same reference configuration.

Task 2. Controlled launch and cancellation

Create a launch file for two rig instances, YAML parameters and an action for a small simulated joint motion. Handle goal, feedback, result and cancel; after cancellation, the simulator must enter the specified stopped state. Validate parameters before activation. Save one launch command and a cancellation scenario. Check: package resources can be found from another working directory; reject an invalid limit or unknown joint name with a clear reason. Endless respawn must not hide errors, and restarting must not resume motion on its own.

Separately compare nodes in different processes with components in one container. At this stage, measure correctness of names and data; assess performance gains from intra-process communication through measurements in the next block.

DDS, QoS and callback execution

Study delivery versus freshness

RMW connects the ROS API to middleware; DDS discovery finds participants, and a domain separates graphs on the chosen network. Record RMW_IMPLEMENTATION, ROS_DOMAIN_ID and network conditions. QoS includes reliability, durability, history and depth, deadline, lifespan and liveliness. Study compatibility between offered and requested profiles: incompatible nodes can appear in the graph without exchanging messages. Reliable mode does not guarantee that a command reaches the application fresh by its deadline.

For a sensor stream, freshness may take priority over complete delivery; motion commands need an explicit sequence number, age and retry policy. A new subscriber must not accidentally replay an old stored command because of transient_local. A deadline event reports a violation of expected periodicity, but the application implements stopping. History depth N limits the message queue but does not itself specify maximum delay. Check the combination of policy and actual behaviour of the chosen RMW.

Task 3. QoS profiles and message loss

A source publishes N numbered measurement packets at a known frequency. Create three compatible profile pairs and one deliberately incompatible pair; add a slow subscriber and a late start. Record transmitted, received, duplicate and missing sequence numbers, message ages and QoS events. Deliverable: a profile → observed behaviour table. Check: count missing packets by seq within one source and session; do not treat a restart as a huge negative counter jump. Error check: a bag recording made with incompatible recorder QoS cannot be considered complete.

Task 4. Callbacks that delay control

Implement a fast telemetry callback and deliberately slow processing or a service. Compare SingleThreadedExecutor, MultiThreadedExecutor with one mutually exclusive group, and a version with separate groups. In C++, measure callback start and end, queue waiting time and publication-to-callback-start delay on a common time scale. Save p50, p95, p99 and maximum distributions and a group diagram. Check: additional threads need not speed up callbacks in one mutually exclusive group; a blocking request from a callback must not cause an unnoticed deadlock.

Repeat the correct configuration with intra-process communication and state the message-transfer method actually supported. Compare copy counts and latency only for the measured configuration: do not treat zero-copy transfer as a property of every ROS2 exchange. Isolating computation helps, but does not itself turn a general-purpose operating system into a hard real-time system.

TF2, URDF/Xacro and the hardware lifecycle

Study coordinate transforms at the correct time

Describe the tree `base_link` → `imu_link` and `base_link` → `hip` → `knee` → `foot`; use separate names for the finger. TF2 stores timestamped transforms and answers queries for a selected instant. The latest available transform may differ from the transform at measurement time. Study static and moving coordinate frames, buffering, timeouts, extrapolation and handling of a missing link. An incorrect coordinate frame or timestamp can cause a spatial error that resembles a filter error.

In URDF/Xacro, distinguish visual, collision and inertial. Mass is in kg, centre of mass in metres and inertia in $\text{kg}\cdot\text{m}^2$; check the tensor's positivity and physical meaning. A detailed mesh may be used for visual, with a simplified mesh for collision. The joint axis and limits must match the CAD model and actuator. `ros2_control` provides hardware state and command interfaces; read and write connect them to the device. A hardware component's lifecycle and an ordinary node's lifecycle are related by the architecture but belong to different objects.

Task 5. Foot position at measurement time

Use the M5 geometry and a test joint trajectory. 1) Create a leg Xacro and a separate IMU coordinate frame. 2) Publish joint states and tree transforms. 3) Transform a foot point into `base_link` at the measurement time specified by timestamp. 4) Compare the result with a hand calculation and kinematics from earlier months. Save the URDF, frame tree and coordinate test. Check: a known zero configuration gives the expected values in metres; a late measurement requests a historical transform. If a transform is missing or stale, produce a diagnosable failure rather than a zero point.

Task 6. Hardware simulator: preventing premature commands

Create a minimal `ros2_control` hardware component with position state and command interfaces for selected joints. Configure checks names and limits; activate enables commands after obtaining an initial state; deactivate puts the simulator in the specified mode. Read returns synthetic feedback; write applies limits and updates a counter. Save the package, controller manager configuration and transition test. Check: no motion is executed before activation; a read or write error triggers the expected state and event. The simulator validates interfaces and logic; check the real driver, delays and physical stopping separately.

Diagnostics, tracing and fault replay

Study tools for observing the software

A log explains an individual event; a diagnostic status describes component state; a metric shows behaviour over an interval; tracing reveals execution order and duration. Monitor IMU and joint-data freshness, hardware-layer state, controller errors, missing-message counters, command age and recording load. Every warning must contain a reason, timestamp and recovery condition. A permanent “OK” line does not provide this information.

Rosbag2 records messages and their timing information. Save parameters, code versions, random-number seeds, hardware events and external clocks separately. Reprocessing a recording does not guarantee the same thread-scheduling order. For a reproducible experiment, fix the initial state, use `use_sim_time`, `/clock` source and input-order rules. Events sharing a timestamp may require a custom seq or a deterministic handler in the test.

Task 7. Recording, dashboard and causal sequence

Record IMU data, joint states, estimates, commands, diagnostics and the required TF. Save parameters and a version manifest. In Plotjuggler or Foxglove, create one dashboard: reference and angle, IMU age, hardware-layer state and controller error. Then delay the source and trace sensor data stale → monitor detects → command blocked. Deliverable: a bag recording, dashboard configuration and a brief report. Check: the original measurement sequence number and every transition are visible; the event does not disappear when switching plots. Compare source and recording counters to assess bag completeness.

Task 8. Automated fault replay

Create a `launch_testing` test: start the system with the simulator, wait for readiness, replay a fixed bag and verify the fault event. Add a timeout and terminate all processes. Separately enable `ros2_tracing` for one callback-delay case if the Linux environment supports the chosen tracing mechanism; state environment limitations. Check the exit code, result contents and system state after the test. If a node fails to become ready or hangs during shutdown, the test must fail rather than wait indefinitely.

Define the required repeatability: matching diagnostic events and values within tolerance, or an identical sequence of processed messages. Do not call a recording deterministic merely because its file is unchanged. State the controlled sources of nondeterminism and any remaining differences in delays measured by physical clocks.

Requirements for the ROS 2 v1.0 system

Packages and responsibilities

robot_interfaces contains only messages, services, actions and unit descriptions. robot_description stores Xacro and meshes. robot_core contains calculations without ROS dependencies; robot_nodes calls them from estimator, controller and monitor nodes. robot_hardware provides the hardware simulator; robot_bringup stores launch and YAML; robot_tests stores integration scenarios; experiments stores version manifests and recordings. Packages may be combined if these responsibility boundaries and the absence of cyclic dependencies are preserved.

Choose one flow: mock_joint/imu → estimate → control_request → hardware. The monitor receives independent freshness indicators and permits or prohibits activation. The command interface has one owner; the diagnostic dashboard does not bypass it to write commands. For every graph edge, record message type, units, QoS, frequency, time source and stale-data behaviour. Namespaces allow a second hand instance to launch without changing source code.

Aligning time

Within a process, measure durations with a monotonic clock. A ROS message timestamp provides a shared time reference and supports TF; across multiple machines, declare the synchronisation method or restrict one-way latency measurement to one machine. Do not subtract clocks from different devices and call the difference transport latency without estimating their offset. Separately measure publication period, callback start, callback duration and data age when a command is calculated.

Changes to system time or /clock must not silently change freshness monitoring. Choose virtual time in advance for replay, and a defined stable time base for hardware freshness checks. When replay pauses, controller behaviour must match the selected mode. Indefinitely repeating an old physical command is unacceptable. Test replay on the simulator using a separate namespace and configuration.

Reproducibility and documentation

One command builds the workspace, another starts the simulator and a third runs checks. State dependencies, tested OS and architecture, terminal procedure and expected topics. The report contains the graph, QoS table, lifecycle diagram, callback measurements, recorded failure and a test link. Building from a clean directory checks paths to installed resources, rather than accidental absolute paths from your computer.

Dashboard snapshots provide a quick overview, but the bag, manifest and calculated metrics remain the primary evidence. The project rests on clear responsibilities: who produces data, who makes decisions and who may operate the actuator.

Six integration and fault tests

Delivery and execution

F1 - incompatible QoS. Launch a publisher and subscriber with conflicting reliability or durability requirements and demonstrate the missing expected exchange. Expected: a diagnostic explanation rather than “the sensor is broken”. Then fix the profile and repeat the same experiment with N packets, saving counters.

F2 - slow callback. Artificial processing takes longer than the source period. Expected: queue size and data age grow, or messages are lost according to the profile; measure p99 and the maximum. Explain how executor and callback-group choices affect the result. An average frequency close to the target does not confirm data freshness.

F3 - source restart. The source resets seq and temporarily disappears. Expected: a new session_id distinguishes restart from counter overflow or packet reordering; freshness monitoring detects the pause; data returning does not automatically activate the device. Record recovery state as a separate event.

Geometry, hardware layer and replay

F4 - TF from the future or past, and an incorrect coordinate frame. Supply one measurement packet outside the available buffer and another with an unknown frame_id. Expected: the failure reason is visible, and no command is generated from an unverified transform. In the corrected case, a known foot point agrees with an independent calculation within the declared tolerance.

F5 - configure, read or write error. Supply an invalid joint limit, then simulate a hardware error in the active simulator. Expected: premature activation is prohibited, the cause is diagnosed, and controllers do not silently keep writing to an invalid interface. Fix the specific sequence of controller and hardware-component actions according to the installed ros2_control version and the test.

F6 - an incomplete bag and a shutdown hang. Remove a required stream or stop one node before the scenario ends. Expected: the replay test detects the missing event or data and ends at its timeout; all processes terminate. A repeated run inherits neither old nodes nor an accidental /clock from the previous test.

Confirming a correction

Each negative test has a corrected counterpart with identical initial conditions. Record the injected error, observed event, detection delay, final state and path to the bag or trace. If the cause remains unknown, leave the status open. Do not substitute increases to every timeout for a correction: any new time limit must be justified by measurements and control requirements.

Virtual-machine special case: participant discovery may depend on network mode, while tracing depends on the kernel and access to system mechanisms. Do not hide these environment limitations under “everything checked”; still complete console and simulator checks, and prepare a separate rerun plan for any unavailable hardware check.

Hours 1-40: the graph and delivery behaviour

Hours 1-20: interfaces and launch

Theory 1-10: 1-2: graph and responsibility; 3-4: topics, services and actions; 5-6: parameters and IDL; 7-8: launch and namespaces; 9-10: node composition within a process and the motion-cancellation plan. These are five two-hour sessions; end each by recording an interface decision for the leg and its reason. Do not study every ROS package at once.

Practice 11-20: 11-12: build a minimal Jazzy workspace and check networking and architecture; 13-14: complete task 1; 15-16: connect nodes to the M11 library; 17-18: complete task 2; 19-20: launch two namespaces and check invalid-parameter and cancellation handling. Save the environment configuration and exact commands for clean reproduction.

Before proceeding, check that you have a graph, an owner for every interface, a working launch and correct cancellation of simulated motion. No hidden dependency on the current directory or an incidentally open terminal should remain.

Hours 21-40: QoS and the executor

Theory 21-30: 21-22: RMW, DDS, domains and discovery; 23-24: reliability, durability and history; 25-26: deadline, lifespan, liveliness and profile compatibility; 27-28: executors and callback groups; 29-30: definitions of measured delays and intra-process communication. Record your prediction for a slow subscriber before running the experiment.

Practice 31-40: 31-32: compare the profiles in task 3; 33-34: add a late subscriber and restart; 35-36: complete task 4; 37-38: compare groups, processes and intra-process communication; 39-40: plot distributions and choose a profile for each stream. Record both the selected option and a rejected configuration with its reason.

Before proceeding, verify that QoS is chosen for each stream's requirements, data age is measured correctly, and deadlock and queue accumulation can be reproduced. Performance claims apply to the measured OS and RMW.

Hours 41-75: geometry, simulator and recording

Hours 41-60: coordinate frames and hardware layer

Theory 41-50: 41-42: TF tree and axes; 43-44: time, buffer and extrapolation; 45-46: URDF/Xacro, masses and collision geometry; 47-48: `ros2_control` state and command interfaces; 49-50: configure, activate, deactivate and errors. End every session by checking one concrete fact: joint sign, metric dimension, timestamp or authority to write a command.

Practice 51-60: 51-52: assemble the leg description; 53-54: complete task 5; 55-56: implement the minimal simulator in task 6; 57-58: connect controller manager and check transitions; 59-60: inject TF, read and write errors and document the diagram. A complex visual model is unnecessary in this block: geometrically correct simple shapes are preferable to an unverified detailed mesh.

Before proceeding, verify a known foot point numerically, check that the hardware simulator does not execute a premature command, and confirm that errors appear in diagnostics. Do not separately claim physical readiness.

Hours 61-75: observation and fault replay

Analysis 61-65: 61-62: `rosbag2`, `/clock` and state during replay; 63-64: diagnostics, logging, tracing and `launch_testing` criteria; 65: compile the final list of supporting results. Split the last five hours as 2 + 2 + 1; preparing for G2 here means collecting results of checks already completed.

Practice 66-75: 66-67: complete task 7; 68-69: save and explain a recorded failure; 70-71: complete task 8 with timeout and process termination; 72-73: run the six negative scenarios; 74-75: clean build, replay and report. If tracing is unavailable, include callback timestamps, an explanation of the environment limitation and instructions for repetition on a suitable Linux system.

Before proceeding, verify that system v1.0 can be reproduced and explains one actual software failure. The material is ready for architectural review and recording replay at G2; the checkpoint's other criteria are accounted for separately.

Acceptance and preparation for a digital twin

Demonstrating the working stack

From a clean environment, build the packages, launch two namespaces and show IMU and joint data flow and action cancellation. Enable a slow callback and explain what happens using metrics. Finish by replaying a fault bag and checking that the automated test detected the expected event. Show RViz, counters, hardware-layer state and the absence of processes left running.

Required acceptance checks: build and launch are reproducible; units and data ownership are documented; QoS is compatible and tested; TF uses measurement time; the simulator handles a fault correctly and enters the specified state. The recording contains required streams, repeatability criteria are explicit, and automated tests terminate on both success and failure. Starting a node alone does not confirm correct command execution.

Self-check

1) When should you use an action instead of a service? 2) Why do parameters not replace a command stream? 3) How does incompatible QoS appear in the graph? 4) Why does reliable not mean fresh? 5) How does deadline differ from application-level freshness monitoring? 6) Why do multiple threads not rescue one mutually exclusive callback group?

7) How do you obtain a transform at measurement time? 8) Why should visual and collision differ? 9) Who owns the command interface? 10) What happens after a read or write error in your particular stack? 11) Which data outside the bag are needed for repetition? 12) Why can the same bag run with different delays? Support answers with configuration, a log or a test.

Handover to M13 and the G2 boundary

Pass on the leg Xacro, joint names, hardware interfaces, launch, QoS, bag template and fault catalogue. This is the framework for a digital twin in Gazebo: the model will replace the hardware simulator while preserving external interfaces and their requirements. Initial quadruped geometry in M13 will extend one validated leg; a physical quadruped is not yet required.

Architectural and replay material is ready for G2, but the overall plan also includes a one-leg HIL criterion developed in M13. This criterion remains open until an experiment with real hardware; a software simulator does not satisfy it. Reaching the G2 calendar date does not justify assigning PASS to every item.

If behind schedule, reduce the dashboard to four plots and retain one hardware simulator. Preserve QoS and execution-time measurements, TF at measurement time and a reproducible failure. If physical-interface validation or tracing is deferred, state the environment, command, expected file and specific untested criterion.

Assigned reading: a focused route through the sections

The references are mainly in English; keep the plan and definitions in English. Read selected sections before the task rather than whole books. Core reading is included in the existing 35 hours of analysis; consult references as needed.

Core reading

E. Renard, ROS 2 from Scratch (2024). [Book and contents](#). Chapter 3 Uncovering ROS 2 Core Concepts; in Chapters 5-9, only sections on custom interfaces, action mechanisms, YAML and launch configuration. For tasks 1-2, list requirements for a stream, a request and a cancellable action. Use Chapters 10-12 selectively for TF and URDF.

K. Lynch, F. Park, Modern Robotics (2017). [Book and teaching sections](#). §3.2.1 Rotation Matrices and §3.3.1 Homogeneous Transformation Matrices. For task 5, construct camera → body → world and its inverse by hand; this checks the meaning of TF rather than repeating the kinematics course.

Reference reading

M. van Steen, A. Tanenbaum, Distributed Systems, 3rd ed. (2017). [Authors' contents](#). In Chapters 3 Processes, 4 Communication and 8 Fault tolerance, select explanations of threads, communication and partial failures. For tasks 3-4 and 8, distinguish delivery, execution, timeliness and recovery; the entire textbook is unnecessary.

Three practical references

[ROS 2 Design: QoS](#) - DDS Quality of Service policies and QoS profiles; task 3, compatibility table. Check values and actual behaviour against Jazzy.

[Jazzy: Using time in tf2](#) - Background and Tasks; task 5, requesting a transform at measurement time and handling its absence.

[ros2_control: Hardware Components](#) - lifecycle and state and command interfaces; task 6, one resource owner and permission to activate.

After reading, answer for your stack: who owns the command, when does a measurement become stale, and what stops actuation after a fault? See page 14 for installation, recording and additional tools.

Terms and definitions

Node. A logical ROS component with its own interfaces; several nodes may execute inside one process.

ROS graph. The collection of nodes and their communication links discovered within a specified domain and accessible network.

Topic. A named, typed message stream; publication generally does not acknowledge command execution by a subscriber.

Service. A request-response interaction for a bounded operation; an action is more suitable for a long motion with cancellation.

Action. A protocol for a long-running task with feedback, a result and a cancellation request that the server must handle.

Parameter. A named node setting with a type and change rules; it does not replace a sequence of control commands.

Namespace. A ROS-name prefix that separates subsystem instances; it does not itself provide network isolation.

DDS and RMW. DDS implements data exchange; the RMW layer connects ROS to a specific middleware implementation.

QoS. A set of message-delivery and storage policies; offered and requested profiles must be compatible for communication.

Reliability. A best effort or reliable delivery policy; retransmission does not guarantee message freshness by the control deadline.

Durability. A policy governing availability of previously published data to late subscribers; transient local requires care for motion commands.

Deadline. The expected maximum interval between updates; a violation event reports a problem and the application implements the response.

Executor. A mechanism that selects ready callbacks and executes them on available threads according to group configuration.

Callback group. A group of callback functions governed by mutual exclusion or permission for concurrent execution.

Deadlock. A state of mutual waiting for resources or events in which participants cannot continue execution.

Timestamp. The numerical time of an event on a specified time scale; measurement time may differ from message-receipt time.

TF2. A buffered tree of coordinate transforms that supports queries for relationships between frames at a specified time.

Lifecycle. Explicit component configuration, activation, deactivation and error states, with permitted transitions between them.

Hardware interface. A contract for reading a resource's state and writing its command; control authority must belong to a defined owner.

Replay. Resupplying recorded data with known timing and configuration; the original computation order may not be reproduced.

What to buy and install this month

USE EXISTING EQUIPMENT

Use the MacBook Pro M4, UTM and Ubuntu 24.04 ARM64 installed in M1, Git, CMake, C++ and Python. A software simulator of the hardware is sufficient for M12; no new onboard computer, motor or sensor is needed. Boards and measurement tools from M4-M7 will be needed later for HIL.

INSTALL NOW

In the Ubuntu guest, install ROS 2 Jazzy Desktop with RViz, colcon, rosdep, vcstool, `ros2_control` and `ros2_controllers`, `tf2`, `robot_state_publisher`, Xacro and `rosbag2`. Start with the [official Jazzy installation guide](#). Record arm64 architecture, package versions, RMW and `ROS_DOMAIN_ID`; do not mix distributions in one shell or workspace.

First run the [ros2_control RRBot demo](#) with simulated hardware. Keep the C++ or Python controller library independent of ROS. For the interface package, keep [IDL: Type system](#) available; define array sizes, strings and units before building.

AS NEEDED

For diagnostic tasks, use [Plotjuggler](#) or an existing graphical analyser. Install [ros2_tracing](#) if Linux supports the chosen mechanism and permissions are available; otherwise save callback measurements and state the tracing limitation. Add [launch_testing](#) to launch processes with a timeout.

INSTALLATION CHECK

1) Build two minimal nodes with colcon, check talker and listener and discovery on the chosen network. 2) Open RViz with the leg TF; test UTM graphics in practice. 3) Activate and deactivate the hardware simulator without spontaneous startup. 4) Record and replay `rosbag2` with timestamp and seq, checking QoS compatibility. Test USB passthrough to the guest before M13 rather than assuming every virtual machine supports it.

Preparation and these checks replace repeated setup and reading within the current 75 hours. An installation page does not mean the software is already installed. Buying a licensed copy of the book is optional if access is available; there are no hardware purchases this month.

Motion fundamentals: the physical meaning of URDF

Required: four descriptions of one link

Which torques will the leg shown in RViz require? Kinematics specifies connections, axes and transforms; visual specifies appearance; collision specifies collision and contact geometry; inertial specifies mass, centre of mass and inertia. Before starting, transform one point between two coordinate frames and explain the distinction between kg and kg·m².

robot_state_publisher computes TF from geometry and joint positions without integrating the effects of forces. ros2_control defines command and state interfaces. The name effort does not turn a position servo into a controllable torque source.

One bar: geometry and inertia from dimensions

Base B is fixed in a right-handed frame: x forward, z upward. The joint at the origin rotates about +y. At $q = 0$, the link points along +x; positive q lowers its endpoint: **p_tip in B = (L cos q, 0, -L sin q)**. For $L = 0.20$ m, $q = 0$ gives (0.20; 0; 0) m and $q = \pi/2$ gives (0; 0; -0.20) m. Mass does not enter this calculation.

A uniform bar has length $L = 0.20$ m along the link's x-axis, width and height $d = 0.02$ m, and $m = 0.30$ kg. The link-frame origin is at the joint; its centre of mass in this frame is (0.10; 0; 0) m. Place the inertia frame at the centre of mass with parallel axes:

$$I_{xx} = m(d^2 + d^2)/12 = 0.000020 \text{ kg}\cdot\text{m}^2;$$

$$I_{yy} = I_{zz} = m(L^2 + d^2)/12 = 0.001010 \text{ kg}\cdot\text{m}^2.$$

Off-diagonal elements vanish by symmetry. In inertial, specify this tensor about the COM and the centre-of-mass origin; also offset the bar's visual and collision by $L/2$.

Inertia about the joint axis is **I_joint = I_yy + m(L/2)² = 0.004010 kg·m²**. The added term accounts for circular COM motion. Substituting I_joint for I_yy while origin is at the COM counts the axis transfer twice and distorts dynamics.

At $q = 0$, gravity (0; 0; -mg) N acts at $x = 0.10$ m. Its moment about +y is $+mgL/2 = +0.2943 \text{ N}\cdot\text{m}$; holding requires actuator torque $-0.2943 \text{ N}\cdot\text{m}$. At $q = \pi/2$, the force lever arm is zero. The sign follows from $\mathbf{r} \times \mathbf{f}$.

Doubling mass without changing shape doubles inertia and gravitational torque; TF at a given q is unchanged. Changing visual changes appearance. Enlarging collision geometry changes contact and clearances but does not establish link strength. The bar checks units; real masses and material distributions come from measurements or CAD.

Two model checks within hours 41-60

1. Select a test link in the leg description; save its URDF and p_tip from manual FK and TF at $q = 0$ and $\pi/2$. The example tolerance is 10^{-6} m for this model. **2.** Double the mass in a copy; use a script to check doubled I and torque and unchanged FK. Save mass, COM, the six I elements and coordinate frames; principal moments of inertia must be positive.

Time: 1 theory hour from 45-46 (URDF, masses, collision geometry), and 1 practice hour each from 51-52 (leg description) and 53-54 (task 5, TF). These check the existing model within the 75 hours. Contact comes in M13; no real actuator commands are sent here.

Read: [Lynch, Park, Modern Robotics §8.2, Part 1](#) - COM and inertia; [ROS urdf tutorial, 07-physics.urdf](#) - visual, collision and inertial structure. Do not copy placeholder inertias from XML: they are calculated here.